Allowance Sentinel: Phishing-Resistant, Time-Bound ERC-20 Token Approvals

Joshua Reneeth Rajaa (40312396), Hariharan Duraisingh (40303001), Meenatchi Sundaravelu (40297341)
INSE 6615 – Blockchain Technologies, Concordia University, Montreal, Canada
Email: joshuajojejo123@gmail.com, hariharan02411@gmail.com, meenatchi8499@gmail.com

***Abstract* - Unlimited ERC-20 token approvals are commonly used in DeFi wallets to avoid repeated confirmations, but they also create a large attack surface for phishing and token-drain attacks. In this project we design and implement an "Allowance Sentinel" smart contract that sits between the user's wallet and decentralized applications. The sentinel converts unlimited approvals into time-bound and amount-limited grants, with a panic switch that can freeze all spending. We implement an ERC- 20 test token and the sentinel contract on the Sepolia testnet, and evaluate their behaviour through a series of experiments: token deployment, approval, grant creation, controlled spending, panic mode, and grant expiry. Our results show that the sentinel can prevent abusive spending while remaining compatible with existing ERC-20 tokens and wallets.**

I. Introduction

Decentralized finance (DeFi) applications rely heavily on the ERC-20 approve/transferFrom pattern. To reduce user friction, wallets often ask users to sign an "unlimited" approval for a given token and dApp. If a malicious contract or compromised dApp obtains this approval, it can drain the wallet's entire token balance without any further confirmations.

In this project, we address this security problem by designing a protective smart contract layer, called the *Allowance Sentinel*. Instead of approving each dApp directly, the user approves the sentinel once, and then creates fine-grained, time-bound grants within the sentinel. Only transactions that match an active grant are allowed to spend tokens.

A. Contributions

Our main contributions are:

- We analyze the security risks of unlimited ERC-20 approvals and summarize typical phishing- based token-drain attacks.
- We design an Allowance Sentinel contract that supports time-bound grants, revocation, and a global panic mode.
- We implement an ERC-20 test token (TestToken) and the sentinel in Solidity and deploy them on the Sepolia testnet using Remix and MetaMask.
- We experimentally demonstrate normal spending, revocation, panic mode, and grant expiry, and benchmark the gas costs of the main operations.

II. Background and Related Work

A. ERC-20 Allowances

The ERC-20 standard defines a fungible token interface including three important functions:

- approve(spender, amount)allows a token owner to authorise a spender.
- allowance(owner, spender)returns the remaining approved amount.
- transferFrom(from, to, amount) lets the spender move tokens on behalf of the owner.

Wallets like MetaMask often default to "Unlimited Approval" to avoid repeated confirmations, which attackers exploit. In practice, many wallet interfaces suggest an approval value close to $2^{256} - 1$, which is effectively unlimited. This means that once a spender is approved, it can withdraw any amount up to the entire token balance of the user without asking again.

B. Phishing and Token-Drain Attacks

Phishing attacks often trick users into interacting with fake dApps, airdrops, or NFT mints that request token approvals. Once the user signs a malicious approval, the attacker can use transferFrom to drain tokens in later transactions.

Examples include fake OpenSea or Uniswap interfaces that silently request unlimited approvals. Many users are not aware that a single approval transaction can grant long-term control over their tokens, and the drain can happen hours or days later.

C. Existing Mitigations

There are several tools and proposals to mitigate approval abuse:

- **Revoke.cash** and similar tools allow users to manually revoke approvals.
- Token approval pages on block explorers (e.g., Etherscan) show current allowances.
- Some Ethereum Improvement Proposals (EIPs) suggest improved allowance handling.

However, these mitigations are mostly *after-the-fact*. Users must actively remember to check and revoke approvals. There is no enforced time limit or spending limit on approvals by default. Our work focuses on enforcing these properties at the smart contract level.

III. System Design

A. Architecture Overview

The Allowance Sentinel introduces a dedicated enforcement layer between the user's wallet and the ERC-20 token contract. Instead of approving third–party dApps directly, the user grants a controlled allowance to the Sentinel, which applies

time-bound and amount-bound constraints before forwarding any spending requests. Figure 1 illustrates the overall data flow and the roles of each component in the system.

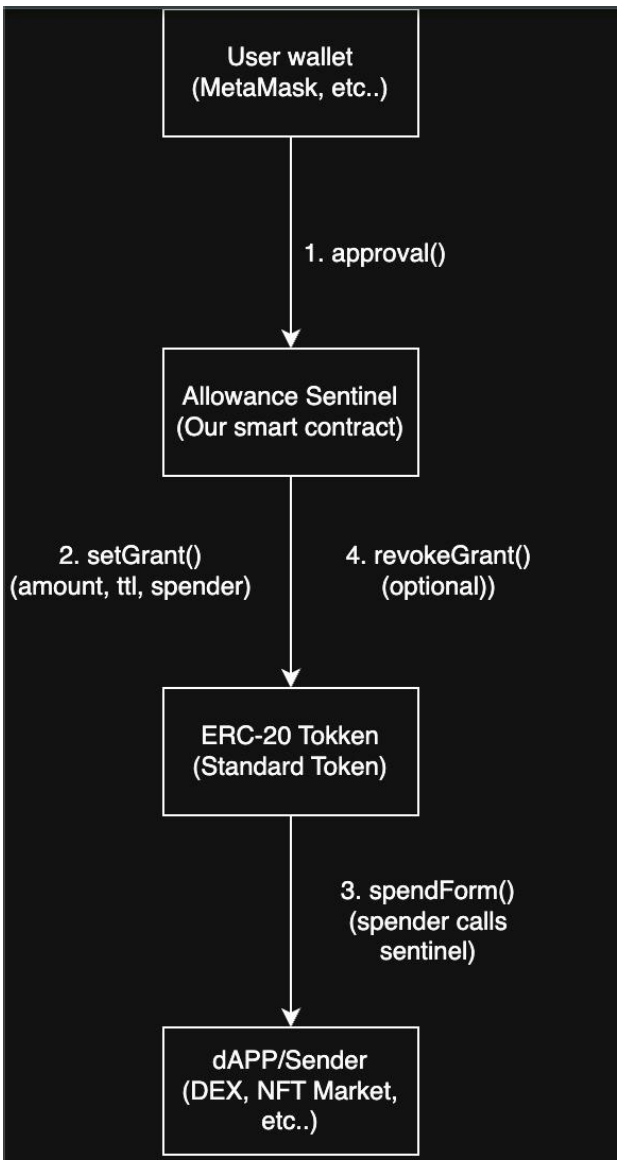


Fig. 1: Overall architecture of the Allowance Sentinel system.

The user approves the Sentinel, initializes a time-bound spending grant, and all spending requests from dApps must pass through the Sentinel before reaching the underlying ERC-20 token contract.

As shown in Figure 1, the workflow begins when a user approves the Sentinel contract instead of a dApp. The user then calls `setGrant()` to define the permitted spendable amount, expiration time, and the allowed spender address. When a dApp attempts to transfer tokens, it must call `spendFrom()` on the Sentinel, which verifies the grant parameters before invoking the ERC-20 `transferFrom()` function. The user may also revoke permissions early using `revokeGrant()`, ensuring full on-chain control throughout the lifetime of the allowance.

The Allowance Sentinel architecture has the following components:

- **User wallet:** holds `TestToken` and owns approvals.
- **TestToken (ERC-20):** standard token contract.
- **Allowance Sentinel:** a contract approved as spender for `TestToken`.
- **Grants:** per-user limits (amount + expiry) stored in the sentinel.
- **Panic mode:** a global boolean that blocks all spending when set.

The high-level flow is:

1) The user deploys `TestToken` and mints an initial balance.
2) The user deploys `AllowanceSentinel`.
3) The user calls `approve(sentinel, amount)` on `TestToken`.
4) The user creates one or more grants using `setGrant`.
5) A dApp (or the user) calls `spendFrom` on the Sentinel.
6) The Sentinel checks the grant and calls `transferFrom` on `TestToken`.

B. *Threat Model*

The Sentinel protects against:

- Phishing contracts trying to overspend token balances using an existing approval.
- Malicious dApps that attempt to spend tokens after an intended time window.
- Excessive spending beyond a configured limit.

It does not protect against:

- Private key theft or full wallet compromise.
- A malicious or buggy Sentinel implementation itself.
- Off-chain social engineering that convinces the user to create overly large grants.

IV. Implementation

A. *Technology Stack*

We used:

- Solidity 0.8.x and Remix IDE for smart contract development.
- MetaMask wallet connected to the Sepolia testnet.
- Etherscan for transaction inspection and gas cost analysis.

B. *TestToken Contract*

`TestToken` is a standard ERC-20 with:

- `name` = `Test Token`
- `symbol` = `TT`
- `decimals` = 18
- `initialSupply` = 100 TT (minted to the deployer)

A simplified constructor is:

```
constructor(uint256 initialSupply) {
    totalSupply = initialSupply * 10 ** decimals;
    balanceOf[msg.sender] = totalSupply;
}
```

Fig. 2: `name()`, `symbol()`, `decimals()`, `totalSupply()` and `balanceOf()` in Remix.

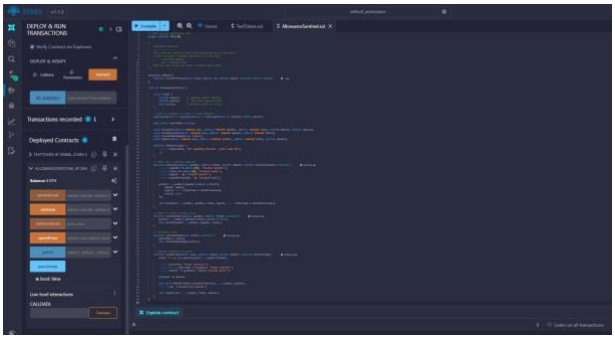


Fig. 3: Deployed `AllowanceSentinel` contract in Remix.

C. Allowance Sentinel Contract

The main data structure is:

```
struct Grant {
    uint256 amount;
    uint256 expiry;
}

mapping(address => mapping(address => Grant))
public grants;
bool public panicMode;
```

Key functions:

- `setGrant(spender, token, amount, validForSeconds)`
- `revokeGrant(spender, token)`
- `setPanicMode(bool)`
- `spendFrom(user, token, amount)`

V. Experimental Setup and Results

All experiments were conducted on the Sepolia testnet. We used one MetaMask account as the user.

A. Token Deployment

We:

1) Compiled `TestToken.sol` in Remix.
2) Selected *Injected Provider – MetaMask*.
3) Deployed the contract and confirmed the transaction in MetaMask.

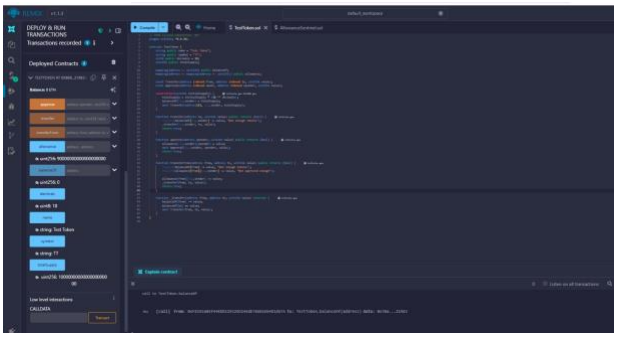


Fig. 4: Deployment of `TestToken` on Sepolia using Remix.

B. Token Information Calls

We verified:

- `name()` returned `Test Token`.
- `symbol()` returned `TT`.
- `decimals()` returned 18.
- `totalSupply()` and `balanceOf(user)` matched 100 TT.

C. Approval to Sentinel

We deployed `AllowanceSentinel`, then called `approve(sentinelAddress, amount)` on `TestToken`. MetaMask displayed a confirmation popup.

The approval transaction was confirmed on Etherscan.

D. Grant Creation and Normal Spending

1) Normal Grant and Spend: We used `setGrant` with:

- `amount` = 10 TT,
- `validForSeconds` = 86400 (1 day).

We then called `spendFrom` with `amount` = 10 TT. The transaction succeeded in Remix and on Etherscan.

2) Panic Mode Experiment: We enabled panic mode via `setPanicMode(true)`. The contract's `panicMode` flag became `true`.

We called another `spendFrom` with the same parameters, the transaction reverted with error "All spending blocked - panic mode ON".

3) Grant Expiry Experiment: We created a new grant with `validForSeconds` = 30 seconds and waited for it to expire. After expiry, calling `spendFrom` resulted in a failure with reason "Grant inactive". We also observed MetaMask gas estimation warnings indicating that the call would likely fail.

E. Gas Cost Benchmarking

Table I shows gas usage for key operations, as reported by Etherscan.

Note: In general, Gas Used ≈ $\frac{\text{Transaction Fee}}{\text{Gas Price}}$.

VI. Comparison With Existing Approaches

Phishing attacks that exploit ERC-20 token approvals are well documented in the Ethereum ecosystem, and several tools have emerged to help users inspect or revoke token allowances. However, most existing approaches remain fundamentally *re-active*. They allow users to clean up approvals *after* an attack

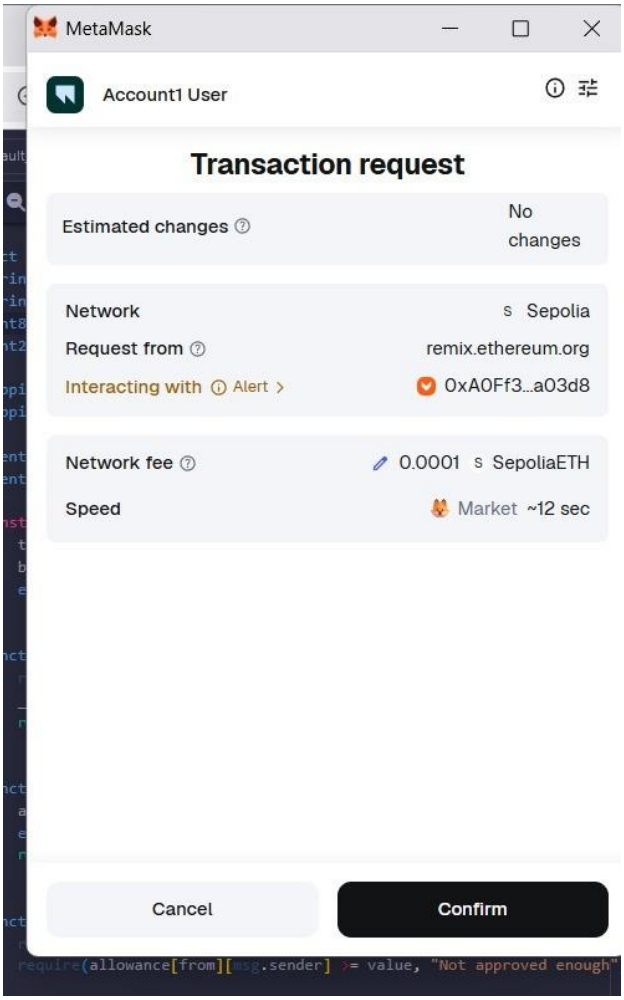


Fig. 5: MetaMask transaction popup when approving the Sentinel as spender.

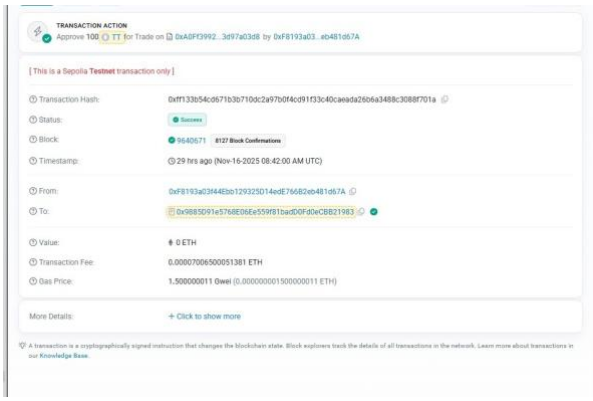


Fig. 6: Approval transaction confirmed on Etherscan (Sepolia).

has already taken place. In contrast, the Allowance Sentinel is a *preventive* mechanism: it modifies the approval workflow itself to limit an attacker's ability to exploit long-lived or infinite approvals.

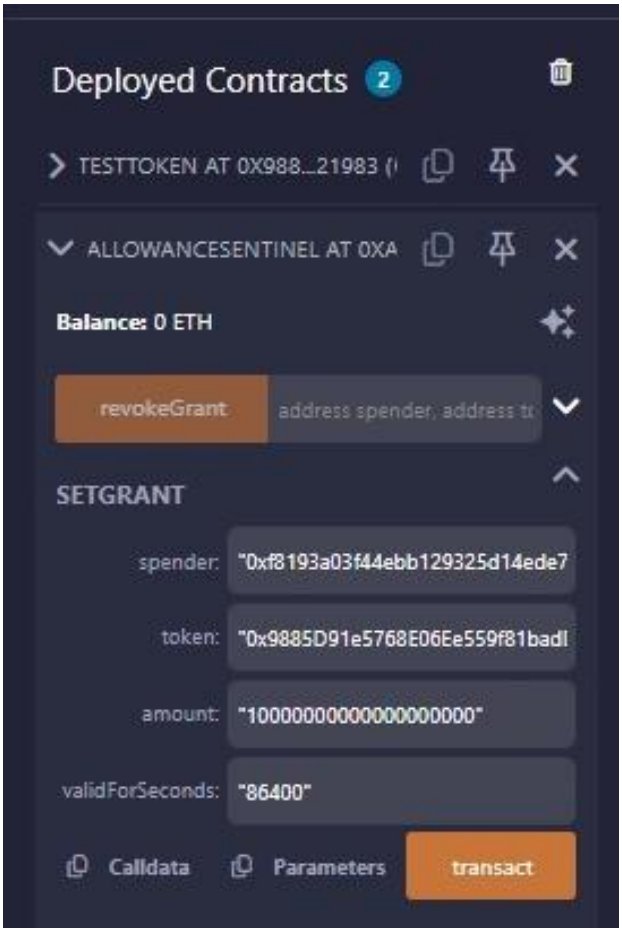


Fig. 7: `setGrant` call for 10 TT over 86400 seconds.

TABLE I: Gas usage for key Allowance Sentinel operations (Sepolia).

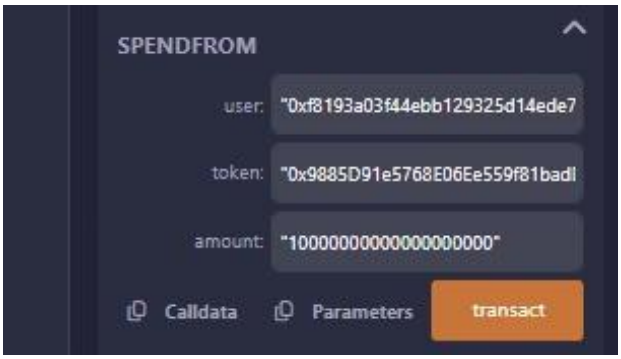
Operation	Gas Used	Outcome
`approve()`	4671	success
`spendFrom()` (normal)	4792	success
`spendFrom()` (panic mode)	2530	revert
`spendFrom()` (expired)	2600	revert

A. Existing Approaches

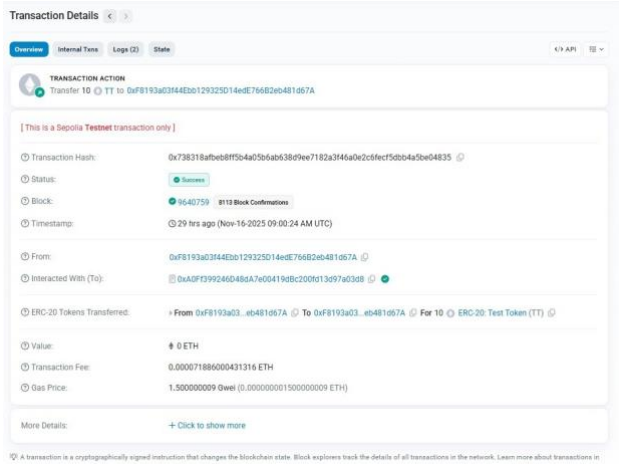
1) 1) Etherscan Token Approval Checker: Etherscan provides a public interface where users can view all ERC-20 approvals associated with their wallet and manually revoke them. Its main strength is visibility: it exposes which contracts currently have spending rights and the maximum amounts they are allowed to transfer.

The main weakness is that the mechanism is entirely reactive. The user must already suspect that something malicious has occurred before checking their approvals. It does not prevent a user from signing a dangerous unlimited approval in the first place.

2) 2) Revoke.cash: Revoke.cash improves usability by integrating directly with wallets and providing a simpler interface for removing or resetting allowances. From a user-experience perspective, it is safer than relying on Etherscan alone.



(a) Successful `spendFrom` call with a valid grant.



(b) Etherscan view of the successful `spendFrom` transfer.

However, the protection is still reactive. If a phishing website tricks a user into providing an unlimited approval, the attacker may drain tokens long before the user thinks to revoke permissions. The core structural weakness of ERC-20 remains unresolved.

3) 3) Wallet Drainer Kits (Adversarial Systems): Malicious toolkits such as *Inferno Drainer*, *Pink Drainer*, and other clones automate phishing attacks by deploying fake dApp front-ends that request unlimited approvals without user awareness.

Because ERC-20 approvals have no expiration and no built-in spending cap, these kits exploit the fact that a single approval can remain valid indefinitely. From an adversarial point of view, their "strength" is the ability to drain tokens hours or even days after the initial phishing event.

These systems highlight a critical design flaw: the ERC-20 approval mechanism was never meant to handle high-risk user interactions at scale.

B. The Allowance Sentinel (This Work)

The Allowance Sentinel directly addresses these structural weaknesses by enforcing:

- a maximum spendable amount,
- a time-to-live (TTL) for each grant,
- an explicit spender identity,
- and an optional global panic switch.

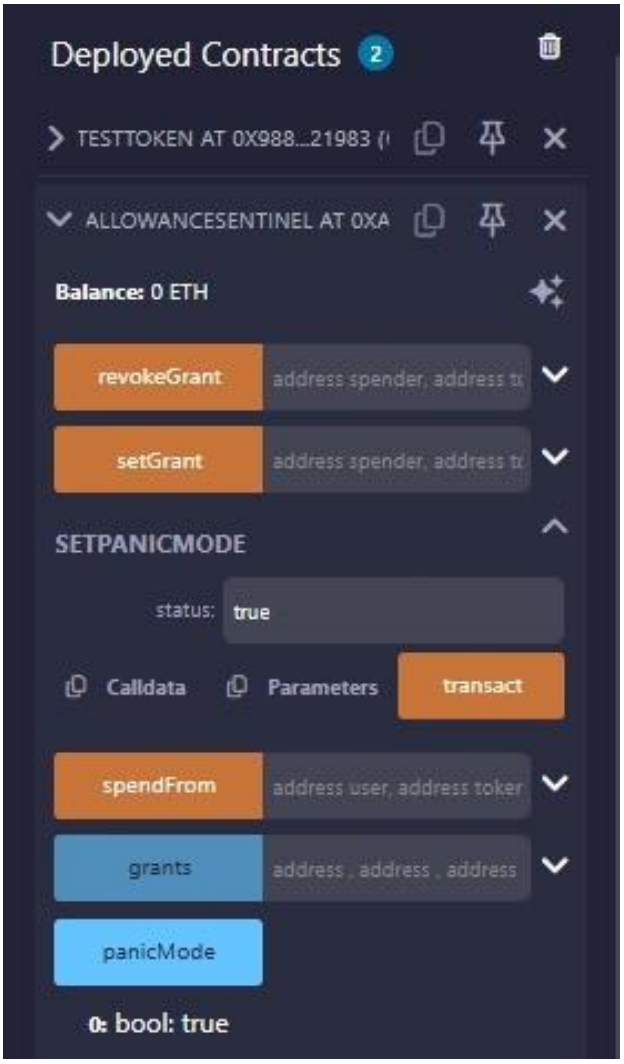


Fig. 9: Panic mode enabled in the Sentinel contract.

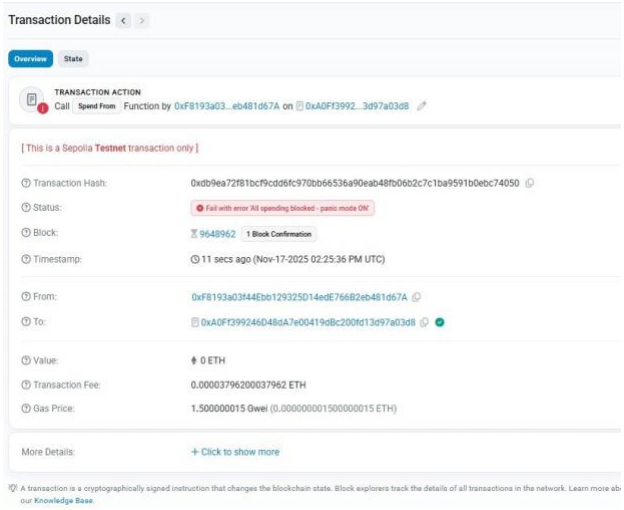


Fig. 10: Etherscan failure showing reason: "All spending blocked - panic mode ON".

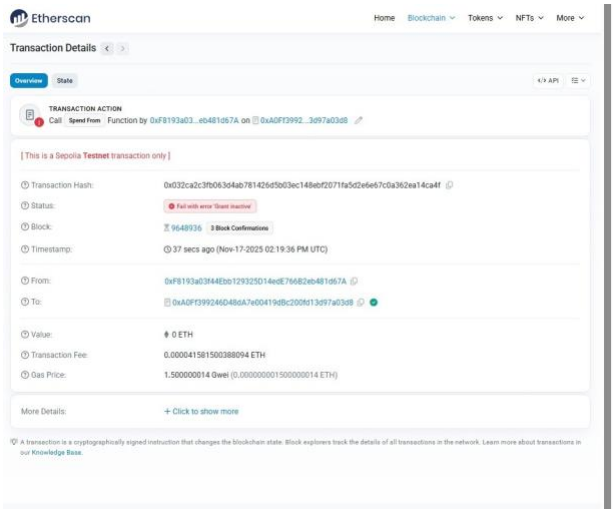


Fig. 11: Etherscan transaction failing with reason "Grant inactive" after expiry.

If a phishing website tricks a user into creating a grant, the attacker cannot:

- spend more than the granted amount, or
- spend any amount after the grant expires.

This transforms infinite-duration approvals into short-lived, low-risk permissions.

The trade-off is that third-party dApps must route spending operations through the Sentinel. While this adds a small gas overhead, it is consistent with safer approval patterns already emerging in industry tools such as Uniswap's `Permit2` and ERC-20 extensions.

VII. Discussion

A. Security Benefits

The Sentinel improves security in several ways.

First, it **bounds potential loss**. In the traditional ERC-20 model, once a user grants an unlimited approval to a malicious contract, the attacker can repeatedly call `transferFrom` until the balance is zero. With the Sentinel, each grant has an explicit `amount` field that is decremented on every successful `spendFrom` call. Even if an attacker gains access to the Sentinel's interface, they can only drain at most the currently remaining grant, not the entire wallet.

Second, the Sentinel introduces **automatic expiry** for approvals. In many real incidents the attacker waits for hours or days after the victim first interacts with the phishing website, to avoid immediate suspicion. By using the `validForSeconds` parameter, the user can ensure that each grant is only usable for a short period of time. After the expiry timestamp passes, the Sentinel will reject further spending, even if the underlying ERC-20 allowance is still non-zero.

Third, the **panic mode** mechanism provides an emergency break. When the user suspects that their wallet or a dApp has been compromised, they can call `setPanicMode(true)` to block all `spendFrom` calls. This allows a single, fast reaction instead of having to revoke approvals one by one for each token and dApp.

Overall, the Sentinel does not claim to completely eliminate approval risks, but it does transform them from an "all-or-nothing" scenario into a more controlled, defence-in-depth model.

B. Usability and Deployment Considerations

Security mechanisms are only useful if users are willing to adopt them. Our prototype is intentionally compatible with existing ERC-20 tokens and wallets: the Sentinel is deployed as a separate contract, and the user interacts with it through standard `approve` and `transferFrom` flows.

From a usability perspective, the main additional step is the explicit grant creation. Instead of approving a dApp directly, the user needs to: (1) approve the Sentinel once, and (2) call `setGrant` before spending. In the current Remix-based interface this looks verbose, but in a dedicated UI the workflow could be simplified. For example, a wallet could expose a single "Pay 10 TT, valid for 1 hour" dialog that internally calls both `setGrant` and `spendFrom`.

Deployment is also flexible. A cautious user might deploy their own instance of the Sentinel, while a wallet provider might deploy a shared, audited Sentinel contract and offer it as a built-in feature. Because the Sentinel does not take custody of tokens and simply forwards `transferFrom` calls, it keeps the trust assumptions close to the standard ERC-20 model.

C. Limitations

Despite these advantages, several limitations remain.

First, users must still **trust the Sentinel contract implementation**. A bug in the grant bookkeeping logic could allow bypassing limits or locking funds. In a production setting the Sentinel should be open-sourced, thoroughly tested, and ideally audited by third-party security firms.

Second, there is an **additional gas overhead**. Each payment now requires an extra contract call through the Sentinel instead of a direct `transferFrom` from the dApp. Our gas measurements in Table I show that this overhead is modest on Sepolia, but high-frequency traders or arbitrage bots may still prefer the cheaper direct-approval model.

Third, the Sentinel **cannot protect against private-key compromise**. If an attacker controls the user's wallet, they can call `setGrant` with a very large amount or disable panic mode. Likewise, the Sentinel cannot stop off-chain social engineering where the victim is persuaded to grant excessive limits.

Finally, our current design only supports a simple mapping keyed by `(user, token)`. In practice, users may want different limits for different dApps using the same token (e.g., 10 TT for a game and 1 TT for a small utility). Supporting such fine-grained policies would require additional indexing by the spender or dApp contract.

D. Future Work

There are several directions for extending this work.

One improvement is to support **per-dApp or per-contract grants**. Instead of having a single allowance per token, the Sentinel could maintain separate grants for each application contract. This would better match how users reason about risk: they may trust a major exchange with a higher limit than a new experimental protocol.

Another direction is to build a **user-friendly dashboard** that visualizes active grants, their remaining amounts, and their expiry times. This could be implemented as an off-chain web interface or as an on-chain view contract consumed by multiple wallets. A dashboard could also suggest recommended defaults, such as small, short-lived grants for new dApps.

Finally, the Sentinel concept could be combined with other proposals such as **permit-style signatures** (EIP-2612) or account-abstraction wallets. For example, a smart-contract wallet could automatically create grants based on daily spending limits, or use risk scores from on-chain analytics to adjust grant sizes dynamically. These integrations would require further design and security analysis but could provide an even smoother user experience.

VIII. Conclusion

Unlimited ERC-20 approvals are a known but still widely exploited weakness in the current DeFi ecosystem. They were introduced for convenience, but in practice they allow a single phishing interaction to give an attacker long-term control over a user's tokens. In this project we explored whether a simple, backwards-compatible smart-contract layer can significantly reduce this risk.

We designed and implemented the Allowance Sentinel, a contract that converts unbounded approvals into explicit, time-bound and amount-limited grants. The Sentinel relies only on the standard ERC-20 interface, so it can be deployed alongside existing tokens without requiring changes to dApp contracts or wallet software. Through our Sepolia experiments we showed that the Sentinel correctly enforces three key behaviours: normal spending under a valid grant, global blocking via panic mode, and automatic rejection of expired grants.

Our measurements indicate that the additional gas overhead of routing payments through the Sentinel is modest for typical user flows. The main remaining cost is cognitive: users must understand the concepts of grants and expiry. We argue that this cost can be reduced by integrating Sentinel-style logic directly into wallets and front-end interfaces, transforming the technical details into intuitive controls such as "spend at most 10 TT within the next hour".

Overall, the Allowance Sentinel does not eliminate all risks related to approvals, but it offers a practical step toward safer defaults. By limiting how much damage a single malicious approval can cause, it moves the ecosystem away from the current "approve everything forever" pattern and closer to a more principle-of-least- privilege design. Future work may extend this foundation with per-dApp policies, better UIs, and integration into account-abstraction wallets.

Appendix A: Use of GenAI Tools

In accordance with the INSE 6615 project policy, we document our use of generative-AI tools in preparing this report.

We used ChatGPT primarily as a *writing assistant* and *explanation helper*. Concretely, we asked the tool to:

- suggest an outline for the report and appropriate section headings that match typical blockchain-security papers;
- rephrase some paragraphs in clearer English while preserving the original technical meaning;
- propose example wording for the abstract, introduction and discussion sections;
- help us check the logical flow between sections and identify missing links (for example, explicitly connecting the experiments to the threat model).

All Solidity smart-contract code, Remix deployments, MetaMask interactions, gas measurements, and screenshots were carried out manually by the authors. We also manually verified that the descriptions of our experiments and results match the actual behaviour observed on the Sepolia testnet.

The final technical content and conclusions reflect our own understanding of the topic. Any mistakes or omissions are our responsibility, not the AI tool's.

References

[1] Ethereum Foundation, "ERC-20 Token Standard," *Ethereum Improvement Proposals, EIP-20*, available at: https://eips.ethereum.org/EIPS/eip-20.

[2] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum Whitepaper, 2014. [Online]. Available: https://ethereum.org/en/whitepaper/.

[3] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf.

[4] OpenZeppelin, "OpenZeppelin Contracts: ERC20 Reference Implementation," Documentation, accessed Nov. 2025. [Online]. Available: https://docs.openzeppelin.com/contracts.